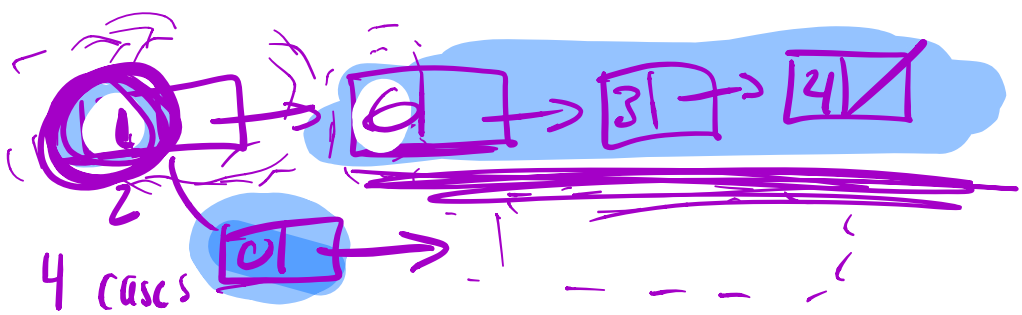
```
(define (decay boxes rotten)
  (cond
  [(null? boxes) nil]
  [(< rotten (car boxes)) (cons (- (car boxes) rotten) (cdr boxes))]
  [else (cons 0 (decay (cdr boxes) (- rotten (car boxes))))]
  )
)
```

boxes: '(3 6 3 4) rotten: 2

return type: Pair



4 cases

1) boxes is nil

2) rotten $\geq$ (car boxes)

3) rotten $<$ (car boxes)

4 cases

boxes are not nil — rotten > 0 / rotten 0

boxes are nil — rotten > 0 / rotten 0

```
def collapse(n):                    left*10 < 10
    left, last = n//10, n%10
    if n < 10 left==0 :
        return n
    elif left %10 == last :
        return last + 10*collapse(n//100)
        return collapse(n//10)
    else:
        return last + 10* collapse(n//10)
                          left
```

```
>>> collapse(12234441)
12341
```

Cases (while n. big)?

1) are the last two digits same?

2) or not?

>>> collapse(9)

10 *

1 2 ~~3~~ ~~4~~ 5

null0 123

1 2 3 ~~4~~

1 2 ~~3~~ ~~4~~ 5

1 2 3

1 2 3 ~~4~~